FLYWEIGHT PATTERN

Structural Design Pattern • Software Engineering

Prepared BY: Team 8

Instructor

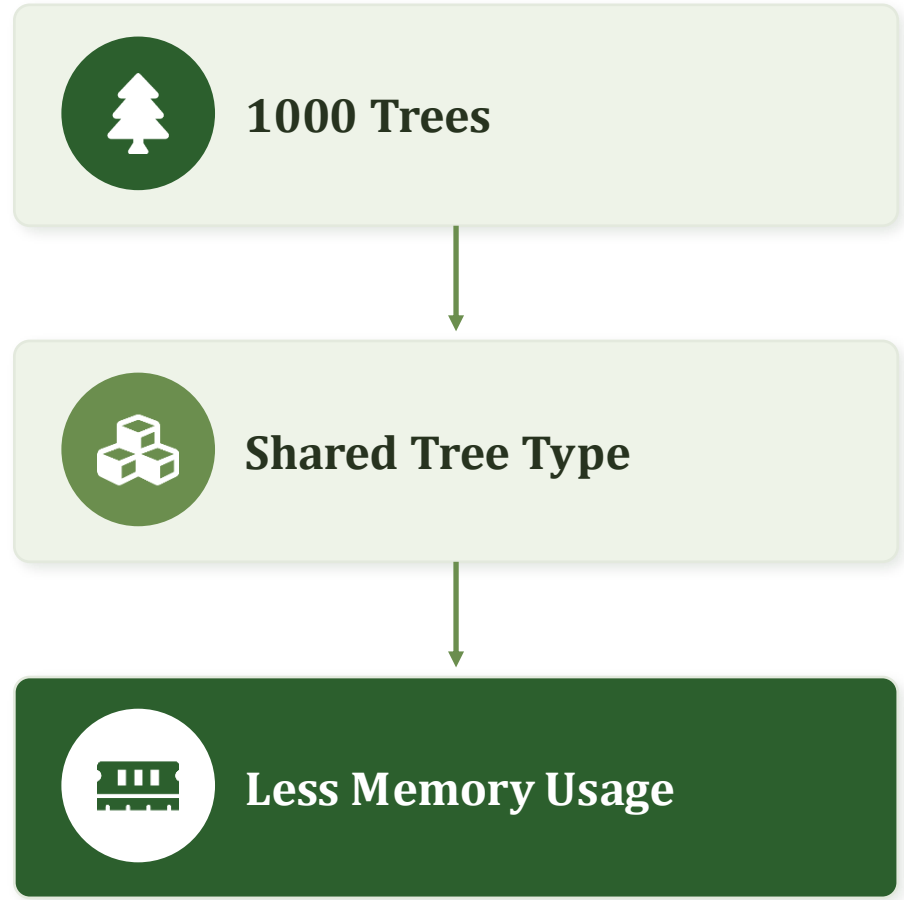
- Md samsuddoha
  Assistant Professor
- Department of CSE
- University of Barishal

ROADMAP

Table of Contents

What is the Flyweight Pattern?

- Flyweight is a Structural Design Pattern.
- It minimizes memory usage by sharing common data among many objects.
- Instead of storing duplicate information repeatedly, shared data is stored only once.
- Objects become lightweight and memory-efficient.



Flyweight = *Share common data to save memory.*

Intent of the Pattern

Use sharing to support a large number of fine-grained objects efficiently.

Memory Optimization

Store common data only once.

Object Sharing

Multiple objects reuse the same state.

Improved Performance

Less memory allocation overhead.

Scalability

Handle thousands or millions of objects efficiently.

Instead of creating duplicate objects, reuse existing ones.

Why This Pattern Is Needed

Applications often create huge numbers of similar objects:

Forest simulation

Game characters

Text editors

Map applications

WITHOUT FLYWEIGHT

```
Tree1 → Color, Texture, Type

Tree2 → Color, Texture, Type

Tree3 → Color, Texture, Type
```

Memory is wasted — identical data repeated.

WITH FLYWEIGHT

```
Shared TreeType  →  Referenced by all
trees
```

Memory usage becomes much lower.

A Forest Game with 100,000 Trees

100,000

TREES

Each tree stores its own copy of:

- Tree Name
- Color
- Texture
- X Position
- Y Position

Huge memory consumption

Slow object creation

Poor scalability

Resource waste

Separate Shared from Unique State

Intrinsic State (Shared)

- Tree Type
- Color
- Texture

Extrinsic State (Unique)

- X Coordinate
- Y Coordinate

Key Features of Flyweight Pattern

Shared State

Common data is reused across many objects.

Reduced Memory Usage

Fewer objects need to be created.

Separation of State

Intrinsic vs Extrinsic state are split apart.

Object Reuse

Existing flyweight objects are reused.

Flyweight Factory

Creates and manages shared objects.

Scalability

Supports very large numbers of objects.

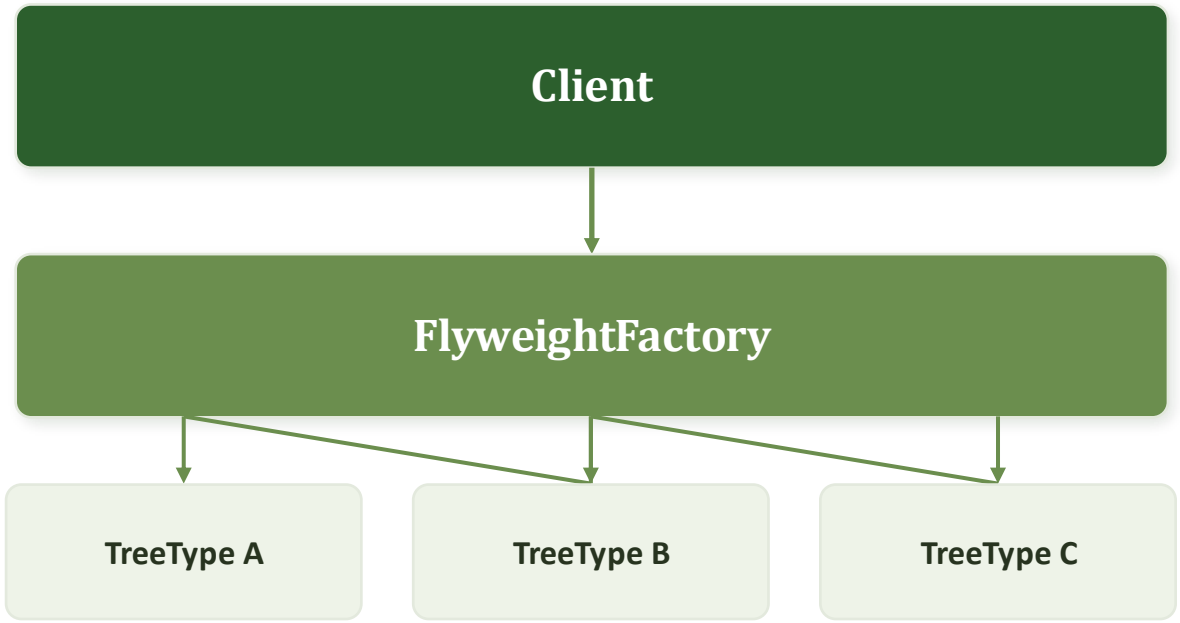
Structure of Flyweight Pattern

1. **Flyweight Interface**
   Declares the operations flyweights must support.

2. **Concrete Flyweight**
   Stores intrinsic (shared) state.

3. **Flyweight Factory**
   Creates and reuses flyweight objects.

4. **Client**
   Maintains extrinsic state and uses flyweights.

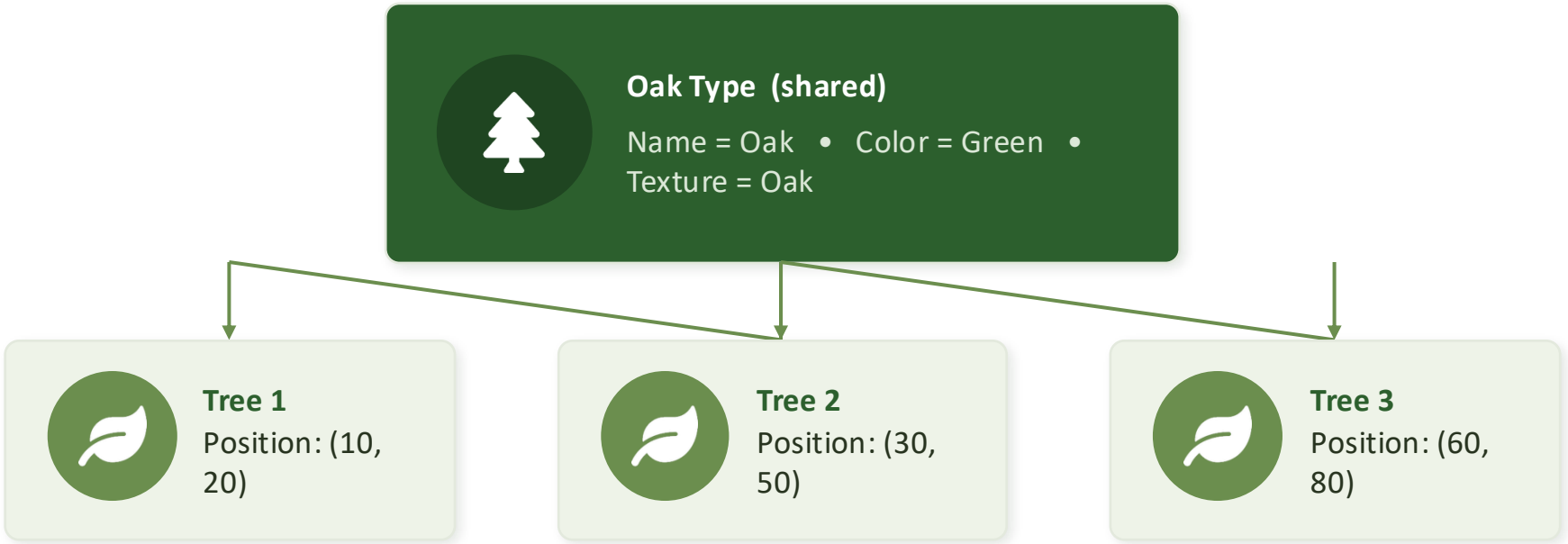


Tree Objects

Store only unique data — e.g. coordinates — and reference a shared flyweight type.

Real-World Analogy: A Forest

Imagine **100,000 Oak Trees** in a forest. All of them share the same Name, Color and Texture — only their positions differ.



Result: one shared Oak object serves thousands of trees.

How Flyweight Works — Step by Step

1. Client requests a flyweight object
2. Factory checks if it already exists
3. If yes, return the existing object
4. If not, create a new flyweight
5. Client stores only unique data
6. Many objects share the same flyweight

FINAL RESULT

- Less Memory
- Faster Execution
- Better Scalability

Pros and Cons

Pros

- Reduces memory usage
- Improves performance
- Supports large-scale applications
- Encourages object reuse
- Better scalability

Cons

- More complex design
- Requires separation of state
- Debugging can be harder
- Not useful for small systems

When to Use vs When Not to Use

Use It When

- Large number of similar objects exist
- Memory consumption is high
- Many objects share common data
- Object creation becomes expensive

Examples: game development, text editors, GIS systems, graphic applications, browser rendering engines.

Avoid It When

- Small number of objects — savings are negligible
- Objects are unique and don't share state
- Simplicity is preferred over flexibility
- Shared state is difficult to identify

Gaming Example: The Forest Game

WITHOUT FLYWEIGHT

100,000

Tree Objects

Each object stores its own copy of:

- Name
- Color
- Texture
- Position

→ Huge memory usage

WITH FLYWEIGHT

3

Shared Tree Types

Reused by all 100,000 trees:

- Oak Type
- Pine Type
- Mango Type

→ Memory reduced dramatically

Coding Example (Java)

FlyweightDemo.java

```
import java.util.HashMap;
import java.util.Map;
import java.util.ArrayList;
import java.util.List;

class TreeType {
    private String name;
    private String color;
    private String texture;

    public TreeType(String name, String color, String texture) {
        this.name = name;
        this.color = color;
        this.texture = texture;
    }
```

FlyweightDemo.java

```
public void draw(int x, int y) {
        System.out.println(
            "Drawing " + name +
            " tree at (" + x + ", " + y + ")" +
            " Color: " + color +
            ", Texture: " + texture
        );
    }
}

class Tree {
    private int x;
    private int y;
    private TreeType type;
public Tree(int x, int y, TreeType type) {
        this.x = x;
        this.y = y;
        this.type = type;
    }

    public void draw() {
        type.draw(x, y);
    }
}
```

FlyweightDemo.java

```
class TreeFactory {
    private static final Map<String, TreeType> treeTypes = new HashMap<>();

    public static TreeType getTreeType(
            String name,
            String color,
            String texture) {

        String key = name + "-" + color + "-" + texture;

        if (!treeTypes.containsKey(key)) {
            treeTypes.put(
                key,
                new TreeType(name, color, texture)
            );
            System.out.println("Creating TreeType: " + key);
        }

        return treeTypes.get(key);
    }

    public static int getTreeTypeCount() {
        return treeTypes.size();
    }
}
```

FlyweightDemo.java

```
class Forest {
    private List<Tree> trees = new ArrayList<>();

    public void plantTree(
            int x,
            int y,
            String name,
            String color,
            String texture) {

        TreeType type =
            TreeFactory.getTreeType(name, color, texture);

        trees.add(new Tree(x, y, type));
    }

    public void draw() {
        for (Tree tree : trees) {
            tree.draw();
        }
    }
}
```

FlyweightDemo.java

```
public class FlyweightForestDemo {
    public static void main(String[] args) {

        Forest forest = new Forest();

        // Plant many trees
        for (int i = 0; i < 1000000; i++) {
            forest.plantTree(
                i,
                i + 10,
                "Oak",
                "Green",
                "OakTexture.png"
            );
        }
```

FlyweightDemo.java

```
    for (int i = 0; i < 1000000; i++) {
            forest.plantTree(
                i,
                i + 20,
                "Pine",
                "Dark Green",
                "PineTexture.png"
            );
        }

        System.out.println(
            "\nNumber of TreeType objects created: "
            + TreeFactory.getTreeTypeCount()
        );
    }
}
```

References

Refactoring Guru

refactoring.guru/design-patterns/flyweight

Gang of Four (GoF)

Design Patterns: Elements of Reusable Object-Oriented Software

Oracle Java Documentation

docs.oracle.com

Video Tutorial

youtu.be/eoY4tB6J2d4

Conclusion

The Flyweight Pattern is a structural design pattern that reduces memory usage by sharing common object data across many fine-grained objects.

- **Saves memory**
- **Improves performance**
- **Supports large-scale applications**
- **Encourages object reuse**
- **Makes systems more scalable**

Best suited for applications containing a large number of similar objects.

Thank You!

Flyweight Pattern • Structural Design Pattern

Any Questions?